OPERATING SYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision.

Q2 – CPU Scheduling — 8 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];
        // TODO 1: Run p for min(quantum, left[p]) time units: print " P(time-newtime)",
        //   advance time, reduce left[p]; add every process that has now arrived and is
        //   not yet added to the queue; then put p back at the end of the queue if it
        //   still has time left, otherwise record finish[p] and count it as done

    }
    double tw = 0, tt = 0;
    printf("\nProcess  TAT  WT\n");
    for (int i = 0; i < N; i++) {
        int tat = finish[i] - arrival[i], wt = tat - burst[i];
        tt += tat;
        tw += wt;
        printf("%-7s %4d %3d\n", name[i], tat, wt);
    }
    printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
    return 0;
}
```

Expected output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21)
P1(21-23) P3(23-26)
Process  TAT  WT
P1        23  15
P2        15  11
P3        24  15
P4        18  13
Average TAT = 20.00, average WT = 13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

Q3 – Threads — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

Q4 – Inter-Process Communication — 6 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results.

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

/* The parent (sensor reader) sends readings to the child (analyser) through a pipe. */
int main(void)
{
    int fd[2];
    if (pipe(fd) == -1) { perror("pipe"); return 1; }
    pid_t pid = fork();
    if (pid == 0) {
        close(fd[1]);                          /* child only reads */
        int r, count = 0, sum = 0, max = 0;
        // TODO 1: Read one int at a time from fd[0] until read returns 0 (the parent
        //   closed the pipe); keep the count, the sum and the maximum

        printf("analyser: %d readings, average %.1f, max %d\n", count,
               count ? (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /* parent only writes */
    int readings[] = {31, 33, 36, 34, 41, 38};
    for (int i = 0; i < 6; i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /* signals end of data to the child */
    waitpid(pid, NULL, 0);
    printf("reader: done\n");
    return 0;
}
```

Expected output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors — 11 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;
    for (int item = 1; item <= ITEMS; item++) {
        // TODO 1: Wait on empty, lock the mutex, put item at buffer[in], advance in
        //   circularly, unlock, then post full

    }
    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;
    for (int k = 0; k < ITEMS; k++) {
        // TODO 2: Wait on full, lock the mutex, take the item at buffer[out], advance
        //   out circularly, unlock, post empty and add the item to consumed_sum

    }
    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_t p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Expected output:

```
items consumed: 20, sum = 210 (expected 210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section.

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

DAY-4 DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program.

```c
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};
int maxm[P][R] = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}};
int alloc[P][R] = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0, 2}};

/* Returns 1 if the state is safe and stores a safe sequence in seq. */
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;
    memcpy(work, avail, sizeof work);
    while (count < P) {
        int found = 0;
        for (int p = 0; p < P; p++) {
            if (finish[p]) continue;
            // TODO 1: If need = maxm[p] - alloc[p] is <= work for every resource, let p
            //   finish: add alloc[p] to work, mark finish[p], append p to seq and set
            //   found = 1

        }
        if (!found) return 0;
    }
    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
        if (req[r] > maxm[p][r] - alloc[p][r] || req[r] > avail[r]) {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    for (int r = 0; r < R; r++) { avail[r] -= req[r]; alloc[p][r] += req[r]; }
    int seq[P];
    if (is_safe(seq)) {
        printf("P%d request: granted, safe sequence", p);
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    } else {
        for (int r = 0; r < R; r++) { avail[r] += req[r]; alloc[p][r] -= req[r]; }
        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];
    if (is_safe(seq)) {
        printf("initial state safe:");
        for (int i = 0; i < P; i++) printf(" P%d", seq[i]);
        printf("\n");
    }
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});
    return 0;
}
```

Expected output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state.

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6

Question	Assessed Area	Bloom's Level	Marks
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend

ANSWERS

Q1 – Processes and Process States

1. Process-state diagram

A process passes through five main states during execution.



Explanation

1. New: Process is created.
2. Ready: Process waits for CPU allocation.
3. Running: Process executes on the CPU.
4. Waiting: Process waits for I/O or an event.
5. Terminated: Process completes execution.
6.

2. Process Control Block (PCB)

PCB is a data structure maintained by the operating system to store information about a process.

PCB component	Description
Process ID	Unique process identification
Process state	Current state of process
Program counter	Address of next instruction
CPU registers	Stores register values
CPU scheduling information	Priority and scheduling details
Memory information	Memory allocation details
I/O status	Open files and I/O devices
Accounting information	CPU usage and execution time

3. C program to create child processes using fork()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
   for (int i = 1; i <= 3; i++)
   {
      pid_t pid = fork();

      if (pid < 0)
      {
         perror("fork failed");
         return 1;
      }

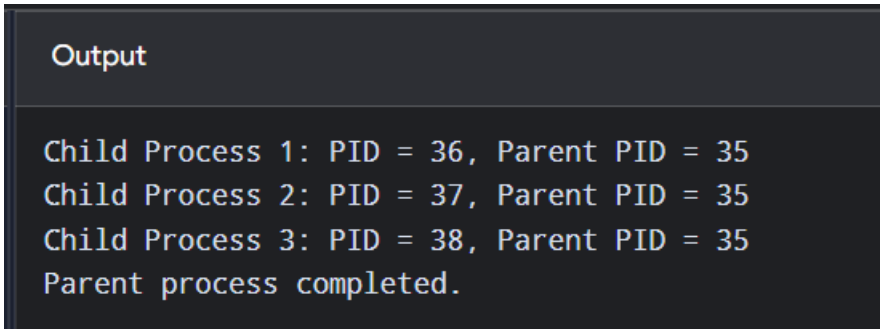
      if (pid == 0)
      {
         printf("Child Process %d: PID = %d, Parent PID = %d\n",
               i, getpid(), getppid());
         exit(0);
      }
   }

   while (wait(NULL) > 0);

   printf("Parent process completed.\n");

   return 0;
}
```

OUTPUT:



4. Evaluation of 2,000 processes

When 2,000 students log in simultaneously:

- Context switching increases CPU overhead.
- Every process requires its own process-control information and memory resources.
- I/O waiting causes processes to remain blocked.
- CPU utilisation may decrease when many processes are waiting for I/O.
- Excessive memory usage can cause performance problems.

5. Design decision

Design	Advantages	Disadvantages
One process per student	Strong isolation	High memory usage
One thread per student	Lower memory overhead	Shared-memory risks
Fixed worker pool	Controlled resource usage	Requires task management

Recommendation: A fixed pool of worker threads is suitable for the online examination server. It reduces resource overhead and handles multiple student sessions efficiently.

6. Alternative concept – Threads instead of processes

If each student session uses a thread:

- Threads share code, data and open files.
- Each thread has its own stack and registers.
- Thread switching generally has lower overhead.
- Memory usage is lower than creating 2,000 separate processes.
- A crash in one thread can affect the entire server process.

Complexity analysis:

Operation	Processes	Threads
Creation	Higher overhead	Lower overhead
Context switching	Generally higher	Generally lower
Memory	Separate address spaces	Shared address space
Isolation	Strong	Limited

Conclusion: A fixed worker pool is better because it balances memory usage, performance and resource management.

Q2 – CPU Scheduling

1. Given process information

The following four jobs are given in the assignment.

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Time quantum for Round Robin = 3

2. Scheduling formulas

Turnaround Time (TAT)

$$TAT = Completion\ Time - Arrival\ Time$$

Waiting Time (WT)

$$WT = TAT - Burst\ Time$$

Response Time (RT)

$$RT = First\ CPU\ Start - Arrival\ Time$$

3. FCFS Scheduling

First Come First Served executes processes in arrival order.

Gantt chart

| P1 | P2 | P3 | P4 |

0 8 12 21 26

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18

Average TAT:

$$\frac{8 + 11 + 19 + 23}{4} = 15.25$$

Average WT:

$$\frac{0 + 7 + 10 + 18}{4} = 8.75$$

4. SJF Scheduling

Shortest Job First selects the process with the smallest burst time among available processes.

Gantt chart

| P1 | P2 | P4 | P3 |

0 8 12 17 26

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9

Average TAT = 14.25

Average WT = 7.75

5. SRTF Scheduling

Shortest Remaining Time First is the preemptive version of SJF.

Gantt chart

|P1| P2 | P4 | P1 | P3 |

0 1 5 10 17 26

Process	CT	TAT	WT	RT
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2

Average TAT = 13.00

Average WT = 6.50

6. Round Robin Scheduling

Each process receives CPU time according to the fixed time quantum.

Gantt chart (quantum = 3)

P1(0-3)
P2(3-6)
P3(6-9)
P4(9-12)
P1(12-15)
P2(15-16)
P3(16-19)
P4(19-21)
P1(21-23)
P3(23-26)

Process	CT	TAT	WT
P1	23	23	15
P2	16	15	11
P3	26	24	15
P4	21	18	13

Average TAT = 20.00

Average WT = 13.50

These values match the expected Round Robin results in your assignment.

7. Priority Scheduling

Priority scheduling executes processes based on priority.

Important: Your assignment does not provide priority values for P1–P4. Therefore, an exact priority Gantt chart and numerical averages cannot be calculated without inventing data.

For an illustration only, if smaller numbers mean higher priority, assign priorities:

Process	Priority
P1	3
P2	1
P3	4
P4	2

These are assumed values, not values from your PDF.

8. Comparison of scheduling algorithms

Algorithm	Average TAT	Average WT
FCFS	15.25	8.75
SJF	14.25	7.75
SRTF	13.00	6.50
RR (q=3)	20.00	13.50
Priority	Depends on priority values	Depends on priority values

9. Effect of Round Robin time quantum

Quantum	Average TAT	Average WT
1	19.00	12.50
3	20.00	13.50
5	17.50	11.00
20	15.25	8.75

Evaluation

- Quantum 1 creates more context switches.
- Quantum 3 provides fair CPU allocation.
- Quantum 5 reduces switching compared with smaller quanta.
- Quantum 20 behaves like FCFS for this job set.
- Very small quantum increases scheduling overhead.
- Very large quantum increases waiting for short jobs behind long jobs.

10. Starvation and ageing

Starvation: A process waits for a long time because other processes continuously receive CPU priority.

Ageing: Gradually increases the priority of waiting processes.

Ageing prevents starvation by ensuring that long-waiting jobs eventually receive CPU time.

11. Multilevel Feedback Queue (MLFQ)

- New jobs enter a high-priority queue.
- Short jobs receive a small time quantum.
- Jobs using their entire quantum move to lower-priority queues.
- Long jobs receive longer time quanta.
- Ageing can promote processes to prevent starvation.

12. Complete the Round Robin C program

```
#include <stdio.h>

#define N 4

int main(void)
{
   const char *name[N] = {"P1", "P2", "P3", "P4"};
   int arrival[N] = {0, 1, 2, 3};
   int burst[N] = {8, 4, 9, 5};

   int quantum = 3;
   int left[N], finish[N];
   int queue[64], head = 0, tail = 0;
   int added[N] = {0};

   for (int i = 0; i < N; i++)
      left[i] = burst[i];

   int time = 0, done = 0;

   queue[tail++] = 0;
   added[0] = 1;

   printf("Gantt: ");
```

```
while (done < N)
{
   int p = queue[head++];

   int run = (left[p] < quantum) ?
            left[p] : quantum;

   int start = time;
   time += run;
   left[p] -= run;

   printf("%s(%d-%d) ", name[p], start, time);

   for (int i = 0; i < N; i++)
   {
      if (!added[i] && arrival[i] <= time)
      {
         queue[tail++] = i;
         added[i] = 1;
      }
   }

   if (left[p] > 0)
      queue[tail++] = p;
   else
   {
      finish[p] = time;
      done++;
   }
}

double tw = 0, tt = 0;

printf("\n\nProcess TAT WT\n");

for (int i = 0; i < N; i++)
{
   int tat = finish[i] - arrival[i];
   int wt = tat - burst[i];

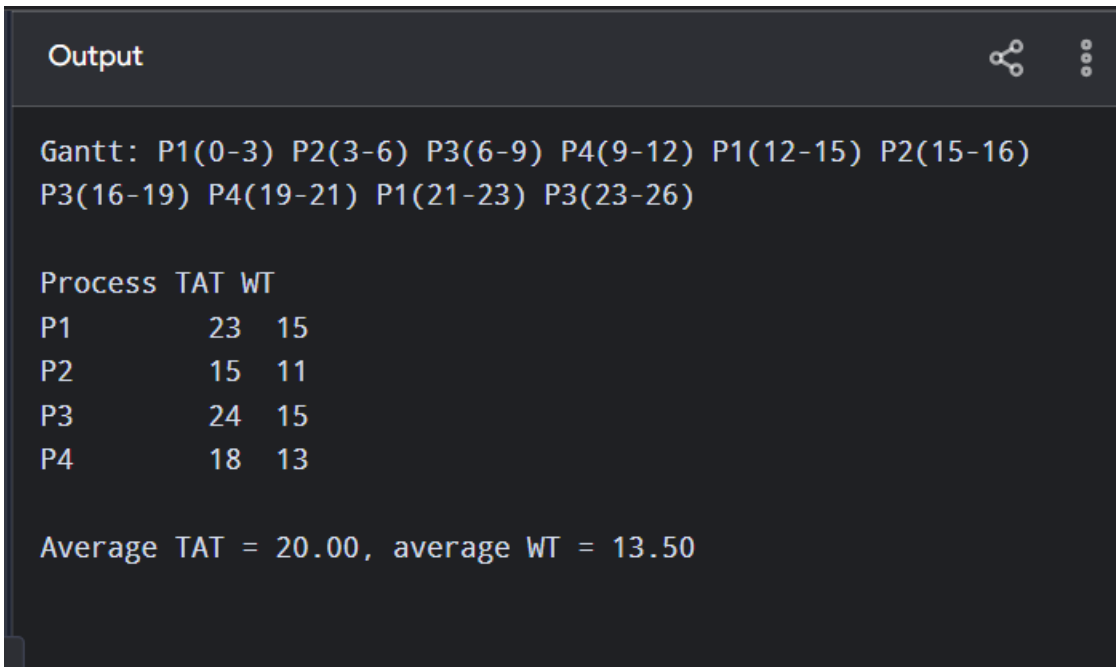
   tt += tat;
   tw += wt;
```

```
        printf("%-7s %4d %3d\n",
               name[i], tat, wt);
    }

    printf("\nAverage TAT = %.2f", tt / N);
    printf(", average WT = %.2f\n", tw / N);

    return 0;
}
```

OUTPUT:



13. Alternative Concept – Multilevel Feedback Queue

If MLFQ is used instead of one Round Robin queue:

- Short jobs receive faster service in higher-priority queues.
- Long jobs move to lower-priority queues.
- Longer time quanta reduce frequent switching for long jobs.
- Ageing prevents starvation.

Complexity analysis:

Algorithm	Scheduling complexity
SJF with linear search	\(O(n)\) per decision
SJF with priority queue	\(O(\log n)\)
RR with circular queue	\(O(1)\) per queue operation
MLFQ	\(O(1)\) per queue operation with suitable queues

Recommendation: For the video-encoding service, MLFQ is a suitable choice because it gives short thumbnail jobs faster service while still allowing long video jobs to execute. SRTF gives the lowest average waiting time for the supplied jobs, but requires burst-time estimates and can starve long jobs.

Q3 – Threads

1. Introduction

Threads are lightweight units of execution within a process. In the traffic-camera system, 64 cameras must process frames simultaneously.
Using multiple threads improves performance compared with processing cameras one after another.

2. Resources shared and not shared

Shared resources	Individual thread resources
Code section	Thread ID
Data section	Stack
Heap memory	CPU registers
Open files	Program counter
Process resources	Thread state

3. User-level and kernel-level threads

User-level threads

- Managed by a user-level thread library.
- Faster to create and manage.
- The kernel may not schedule individual threads separately.

Kernel-level threads

- Managed by the operating system.
- Can run on multiple CPU cores.
- Require kernel involvement for management.

4. Thread models

Model	Description
Many-to-One	Many user threads map to one kernel thread
One-to-One	Each user thread maps to one kernel thread
Many-to-Many	Many user threads map to multiple kernel threads

Evaluation: One-to-one is commonly used in modern operating systems because threads can execute in parallel on multiple CPU cores.

5. C program – Thread pool experiment

```
#include <stdio.h>
#include <pthread.h>
#include <time.h>
#include <stdint.h>

#define CAMERAS 64
#define WORK 300000

volatile unsigned long long result[CAMERAS];

void process_camera(int id)
{
   unsigned long long sum = 0;

   for (int i = 1; i <= WORK; i++)
      sum += ((unsigned long long)i * (id + 1)) % 997;

   result[id] = sum;
}

void *worker(void *arg)
{
   int *range = (int *)arg;

   for (int i = range[0]; i < range[1]; i++)
      process_camera(i);

   return NULL;
}

double seconds(clock_t start, clock_t end)
```

```
{
    return (double)(end - start) / CLOCKS_PER_SEC;
}

int main(void)
{
    clock_t start, end;

    start = clock();

    for (int i = 0; i < CAMERAS; i++)
        process_camera(i);

    end = clock();

    double sequential = seconds(start, end);

    printf("Sequential time: %.6f seconds\n", sequential);

    int pools[] = {4, 8, 16};

    for (int k = 0; k < 3; k++)
    {
        int n = pools[k];
        pthread_t threads[16];
        int ranges[16][2];

        start = clock();

        for (int i = 0; i < n; i++)
        {
            ranges[i][0] = i * CAMERAS / n;
            ranges[i][1] = (i + 1) * CAMERAS / n;

            pthread_create(&threads[i], NULL,
                           worker, ranges[i]);
        }

        for (int i = 0; i < n; i++)
            pthread_join(threads[i], NULL);

        end = clock();
```

```
        double parallel = seconds(start, end);

        printf("%d threads: %.6f seconds\n",
               n, parallel);

        if (parallel > 0)
            printf("Speedup: %.2f\n",
                   sequential / parallel);
    }

    return 0;
}
```

OUTPUT:

```
Output

Sequential time: 0.031707 seconds
4 threads: 0.046591 seconds
Speedup: 0.68
8 threads: 0.049467 seconds
Speedup: 0.64
16 threads: 0.040052 seconds
Speedup: 0.79
```

Output

The execution times and speed-ups depend on your computer. Record the actual output.

Method	**Execution time**	**Speed-up**
Sequential	Actual output	1.00
4 threads	Actual output	Sequential / parallel
8 threads	Actual output	Sequential / parallel
16 threads	Actual output	Sequential / parallel

Speed-up formula:

$$Speedup = \frac{Sequential\ Time}{Parallel\ Time}$$

6. Why speed-up stops increasing

- CPU core count is limited.
- Thread creation introduces overhead.
- Threads compete for CPU and memory.
- Memory bandwidth becomes a bottleneck.
- Synchronisation overhead increases.
-

7. Shared counter problem

If two threads update a shared counter without protection, a race condition may occur.

For example:

counter++;

This operation involves reading, increasing and writing the value. Two threads may interfere with each other.

A mutex or atomic operation is required to protect the counter.

8. Alternative Concept – Processes instead of threads

If separate processes are used for the 64 cameras:

- Each process has its own address space.
- Process creation requires more resources.
- Context switching may have higher overhead.
- Results must be combined using IPC.
- Threads can share results directly through common memory.

Factor	**Processes**	**Threads**
Memory	Separate	Shared
Communication	IPC	Shared variables
Creation	Higher overhead	Lower overhead
Isolation	Strong	Lower
Performance	More overhead generally	Efficient for shared workloads

Complexity analysis:

- Processing all cameras: \(O(64 \times W)\), where W is the frame workload.
- Additional result storage: \(O(64)\).
- Thread-pool processing has approximately \(O(64W/k)\) ideal workload per worker for k workers, excluding overhead.

Conclusion: A thread pool of 8 or 16 threads is worth testing. The best size depends on available CPU cores and measured execution time.

Q4 – Inter-Process Communication

1. Introduction

Inter-Process Communication (IPC) allows two or more processes to exchange information.
In the weather-station system:

- Parent process reads sensor values.
- Child process analyses the readings.
- Both processes exchange data through IPC.

2. Pipe implementation

A pipe provides communication between processes.
Working:

1. The parent creates a pipe.
2. The parent creates a child using fork().
3. The child closes the writing end.
4. The parent sends sensor readings.
5. The child reads and analyses the readings.
6. The parent closes the writing end.
7. The child receives EOF and completes execution.

3. Complete the Code – Pipe

```
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void)
{
```

```
int fd[2];

if (pipe(fd) == -1)
{
   perror("pipe");
   return 1;
}

pid_t pid = fork();

if (pid < 0)
{
   perror("fork");
   return 1;
}

if (pid == 0)
{
   close(fd[1]);

   int r, count = 0, sum = 0, max = 0;

   while (read(fd[0], &r, sizeof(r)) > 0)
   {
      count++;
      sum += r;

      if (count == 1 || r > max)
         max = r;
   }

   printf("analyser: %d readings, average %.1f, max %d\n",
          count,
          count ? (double)sum / count : 0.0,
          max);

   close(fd[0]);
   exit(0);
}

close(fd[0]);
```

```
    int readings[] = {31, 33, 36, 34, 41, 38};

    for (int i = 0; i < 6; i++)
    {
        if (write(fd[1], &readings[i],
                sizeof(readings[i])) < 0)
        {
            perror("write");
            break;
        }
    }

    close(fd[1]);

    waitpid(pid, NULL, 0);

    printf("reader: done\n");

    return 0;
}
```

OUTPUT:

```
Output

analyser: 6 readings, average 35.5, max 41
reader: done
```

4. Why is close(fd[1]) important?

The parent must close the writing end after sending all readings.
When all writing ends are closed, the reader receives EOF (read returns 0).
If the parent does not close fd[1], the child may continue waiting because it cannot know that no more readings will arrive.

5. Shared memory and message passing

Pipe	**Shared memory**
Uses kernel-managed buffer	Processes share a memory region
Data passes through kernel	Processes access common memory
Synchronisation through pipe operations	Requires explicit synchronisation
Suitable for simple communication	Suitable for large data transfers

Data copying:

- Pipes generally copy data into and out of the kernel buffer.
- Shared memory avoids repeated kernel-mediated data copying after the memory is mapped.
- Shared memory still requires synchronisation to avoid race conditions.

6. Alternative Concept – Shared memory with semaphore

A shared-memory solution can be designed using:

1. shm_open() to create shared memory.
2. ftruncate() to set its size.
3. mmap() to map it into both processes.
4. A semaphore to control access.
5. The parent writes sensor readings.
6. The child reads and analyses the readings.

Pseudocode:

Create shared memory
Initialise semaphore

Parent:
  Wait for free space
  Write sensor reading
  Signal data available

Child:
  Wait for data available
  Read sensor reading
  Signal free space

Close and unlink shared memory

7. Growth investigation

Measure the transfer time for different data sizes.

Data size	Pipe time	Shared memory time
1 MB	Measure	Measure
10 MB	Measure	Measure
100 MB	Measure	Measure

8. Complexity analysis

For n readings:

- Pipe transfer: \(O(n)\) time and \(O(n)\) total transferred data.
- Shared memory: \(O(n)\) time to write and read all readings.
- Pipe requires kernel buffering.

- Shared memory requires additional synchronisation.

Conclusion: Pipes are suitable for simple weather-station communication. Shared memory is preferable for large, continuous data transfers when synchronisation is implemented correctly.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors

1. Critical-section problem

A critical section is a part of a program where shared resources are accessed. In the railway ticket-booking system, multiple agents may try to book the same seat simultaneously. Without synchronisation, the same seat may be booked twice.

A correct solution must satisfy three conditions:

1. Mutual Exclusion: Only one process can execute its critical section at a time.
2. Progress: If no process is in the critical section, a waiting process must be allowed to enter.
3. Bounded Waiting: Every waiting process must get an opportunity to enter within a limited time.

2. Peterson's solution

Peterson's solution provides mutual exclusion between two processes.

It uses:

- flag[2] – indicates whether a process wants to enter the critical section.
- turn – indicates whose turn it is.

Algorithm

Process P0:
flag[0] = true
turn = 1

while (flag[1] && turn == 1);

Critical Section

flag[0] = false
Process P1:
flag[1] = true
turn = 0

while (flag[0] && turn == 0);

Critical Section

flag[1] = false

Evaluation

- Ensures mutual exclusion for two processes under the algorithm's memory assumptions.
- Provides progress.
- Provides bounded waiting.
- Uses busy waiting.
- May fail on modern processors if memory ordering is not enforced.
- Requires suitable atomic operations and memory-ordering guarantees in modern C implementations.
-

3. Binary and counting semaphores

A semaphore is a synchronisation variable used to control access to shared resources.

Binary semaphore	**Counting semaphore**
Value is 0 or 1	Value can be 0 or greater
Controls one shared resource	Controls multiple identical resources
Used for mutual exclusion	Used for resource counting

Semaphore operations

- wait() decreases the semaphore value or blocks the process.
- signal() increases the semaphore value and may wake a waiting process.

Seat-counter example

A mutex protects the seat counter.

```
pthread_mutex_lock(&lock);

if (available_seats > 0)
   available_seats--;

pthread_mutex_unlock(&lock);
```

This prevents multiple agents from modifying the counter simultaneously.

4. Classical synchronisation problems

A. Producer-consumer problem

- Producer generates data.
- Consumer removes data.
- Buffer has limited capacity.
- empty semaphore counts empty slots.

- full semaphore counts occupied slots.
- Mutex protects the buffer.

B. Readers-writers problem

- Multiple readers can read simultaneously.
- Only one writer can modify the shared data.
- Writers require exclusive access.
- Semaphores control reader and writer access.

C. Dining-philosophers problem

- Five philosophers share five forks.
- Each philosopher needs two forks to eat.
- Deadlock can occur if all philosophers hold one fork and wait for another.

Solution: Allow only four philosophers to compete for the five forks simultaneously, or use an ordered resource-acquisition strategy.

5. Monitors and condition variables

A monitor is a high-level synchronisation mechanism that combines shared data and operations into one structure.

Condition variables provide:

- wait() – suspends a thread until a condition is satisfied.
- signal() – wakes a waiting thread.

Advantages:

- Easier to understand.
- Reduces synchronisation errors.
- Supports mutual exclusion.
- Makes shared-resource management simpler.
-

6. Complete the Code – Bounded Buffer using Semaphores

This program implements one producer and one consumer, as in the supplied assignment code.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in = 0, out = 0;

sem_t empty, full;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

long consumed_sum = 0;

void *producer(void *arg)
{
   (void)arg;

   for (int item = 1; item <= ITEMS; item++)
   {
      sem_wait(&empty);

      pthread_mutex_lock(&lock);

      buffer[in] = item;
      in = (in + 1) % SIZE;

      pthread_mutex_unlock(&lock);

      sem_post(&full);
   }

   return NULL;
}

void *consumer(void *arg)
{
   (void)arg;

   for (int k = 0; k < ITEMS; k++)
   {
      sem_wait(&full);

      pthread_mutex_lock(&lock);

      int item = buffer[out];
      out = (out + 1) % SIZE;

      pthread_mutex_unlock(&lock);

      sem_post(&empty);
```

```
        consumed_sum += item;
    }

    return NULL;
}

int main(void)
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_t p, c;

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);

    sem_destroy(&empty);
    sem_destroy(&full);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

OUTPUT:

```
Output

items consumed: 20, sum = 210 (expected 210)
```

7. Test with 1 producer and 1 consumer

The above program uses:

- One producer.
- One consumer.
- Five buffer positions.
- Twenty items.

The expected sum is:

\[\frac{20(20+1)}{2}=210 \]

8. Test with 4 producers and 4 consumers

Use the following additional program to test multiple producers and consumers.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 10
#define NP 4
#define NC 4
#define ITEMS 20
#define TOTAL (NP * ITEMS)

int buffer[SIZE], in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

long consumed_sum = 0;

void *producer(void *arg)
{
   int id = *(int *)arg;

   for (int i = 1; i <= ITEMS; i++)
   {
      int item = id * ITEMS + i;

      sem_wait(&empty);
      pthread_mutex_lock(&lock);

      buffer[in] = item;
```

```
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int i = 0; i < TOTAL / NC; i++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&lock);

        int item = buffer[out];
        out = (out + 1) % SIZE;

        consumed_sum += item;

        pthread_mutex_unlock(&lock);
        sem_post(&empty);
    }

    return NULL;
}

int main(void)
{
    pthread_t p[NP], c[NC];
    int id[NP];

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    for (int i = 0; i < NP; i++)
    {
        id[i] = i;
        pthread_create(&p[i], NULL, producer, &id[i]);
```

```
    }

    for (int i = 0; i < NC; i++)
        pthread_create(&c[i], NULL, consumer, NULL);

    for (int i = 0; i < NP; i++)
        pthread_join(p[i], NULL);

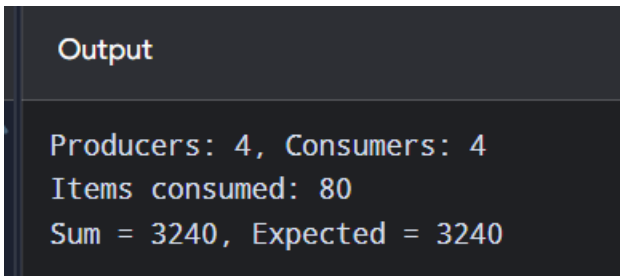
    for (int i = 0; i < NC; i++)
        pthread_join(c[i], NULL);

    printf("Producers: %d, Consumers: %d\n", NP, NC);
    printf("Items consumed: %d\n", TOTAL);
    printf("Sum = %ld, Expected = %d\n",
           consumed_sum, TOTAL * (TOTAL + 1) / 2);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&lock);

    return 0;
}
```

OUTPUT:



Every execution should produce a sum of 3240.

9. What happens if the mutex is removed?

- Multiple producers may write to the same buffer position.
- Multiple consumers may read the same item.
- The in and out indexes may be corrupted.
- Race conditions may occur.
- Items may be lost or consumed more than once.

The semaphore controls buffer capacity, but the mutex protects the shared buffer indexes and data.

10. Deployment decision − Ticket-booking locks

Locking method	Advantages	Disadvantages
One global lock	Simple implementation	High contention
One lock per train	Good concurrency	Some contention within a train
One lock per coach	Higher concurrency	More complex management

Recommendation: One lock per train is a good starting design because it prevents duplicate bookings while allowing different trains to be booked simultaneously. Per-coach locking can provide higher throughput if seat-allocation rules and lock ordering are carefully designed.
Measure each design using identical booking workloads before selecting the final implementation.

11. Alternative Concept − Atomic operations and spin locks

```
#include <stdio.h>
#include <stdatomic.h>
#include <pthread.h>

atomic_int counter = 0;

void *update(void *arg)
{
   (void)arg;

   for (int i = 0; i < 10000; i++)
      atomic_fetch_add(&counter, 1);

   return NULL;
}

int main(void)
{
   pthread_t t[4];

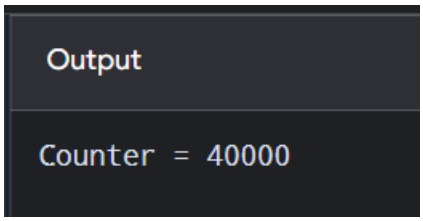
   for (int i = 0; i < 4; i++)
      pthread_create(&t[i], NULL, update, NULL);

   for (int i = 0; i < 4; i++)
      pthread_join(t[i], NULL);

   printf("Counter = %d\n",
         atomic_load(&counter));
```

```
    return 0;
}
```

OUTPUT:



Comparison

Method	Advantage	Disadvantage
Semaphore and mutex	Suitable for longer critical sections	May involve blocking overhead
Atomic operation	Efficient for simple updates	Not suitable for complex operations
Spin lock	Avoids sleeping for short waits	Wastes CPU while spinning

Complexity analysis

- Atomic counter update: \(O(1)\) per update.
- Mutex-protected counter update: \(O(1)\) per update, excluding waiting.
- Spin lock: \(O(1)\) for an uncontended acquisition; contention can increase waiting time.
- Total n updates: \(O(n)\).

Evaluation: Atomic operations are suitable for simple counters. Mutexes and semaphores are better for longer critical sections because spinning wastes CPU time.

Q6 – Deadlock Handling and the Banker's Algorithm

A. Characterise Deadlock

1. Definition

Deadlock is a condition in which two or more processes wait indefinitely for resources held by one another.

In the laboratory equipment system, jobs may require GPUs, software licences and high-speed storage.

2. Four necessary conditions

1. Mutual Exclusion

A resource can be used by only one process at a time.

Example: One GPU is allocated to one job.

2. Hold and Wait

A process holds one resource while waiting for another resource.

Example: A job holds a GPU and waits for a software licence.

3. No Preemption

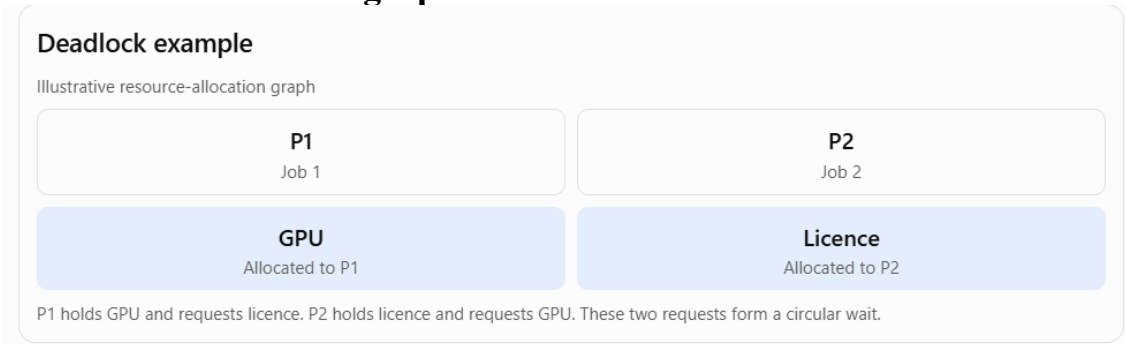
A resource cannot be forcibly removed from a process.

Example: A running simulation cannot have its allocated resource taken away without proper recovery.

4. Circular Wait

A circular chain of processes exists in which each process waits for a resource held by the next process.

3. Resource-allocation graph



Cycle:

$$P1 \rightarrow Licence \rightarrow P2$$

$$P2 \rightarrow GPU \rightarrow P1$$

This creates a deadlock when each resource has one instance.

B. Compare Deadlock Handling Strategies

There are four main deadlock-handling strategies.

Strategy	Explanation	Advantage	Disadvantage
Prevention	Removes one necessary condition	Prevents deadlock	Resource utilisation may decrease
Avoidance	Checks whether allocation is safe	Avoids unsafe allocation	Requires maximum demand
Detection and Recovery	Detects deadlock and recovers	Allows flexible allocation	Recovery may lose work
Ostrich Approach	Ignores deadlock	Simple implementation	Deadlock may stop the system

Evaluation

1. Prevention

The system prevents circular waiting by imposing a fixed resource-allocation order.

2. Avoidance

The Banker's algorithm checks whether resource allocation keeps the system in a safe state.

3. Detection and Recovery

The system periodically detects deadlock and terminates or restarts selected jobs.

4. Ostrich Approach

The system ignores deadlock because handling it may introduce additional overhead.

Recommendation: Deadlock avoidance is suitable for the laboratory equipment scheduler because GPUs, software licences and storage are critical shared resources.

C. Apply the Banker's Algorithm (4 Marks)

1. Given information

Number of processes = 5

Number of resource types = 3

Available resources:

Resource	Available
GPU (A)	3
Licence (B)	3
Storage (C)	2

2. Allocation matrix

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

3. Maximum matrix

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

4. Need matrix

Formula:

$$ Need=Maximum-Allocation $$

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1
P4	4	3	1

5. Safe sequence calculation

Initial available:

$$ Work=(3,3,2) $$

- P1 can finish. New Work = (5,3,2).
- P3 can finish. New Work = (7,4,3).
- P4 can finish. New Work = (7,4,5).
- P0 can finish. New Work = (7,5,5).
- P2 can finish. New Work = (10,5,7).

Therefore:

Safe state

$$ P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2 $$

6. Resource-request algorithm

The request must satisfy:

1. Request must be less than or equal to Need.
2. Request must be less than or equal to Available.

3. Allocate resources temporarily.
4. Check the safety algorithm.
5. Grant the request only if the resulting state is safe.

7. Complete C program – Banker's Algorithm

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};

int maxm[P][R] = {
   {7, 5, 3},
   {3, 2, 2},
   {9, 0, 2},
   {2, 2, 2},
   {4, 3, 3}
};

int alloc[P][R] = {
   {0, 1, 0},
   {2, 0, 0},
   {3, 0, 2},
   {2, 1, 1},
   {0, 0, 2}
};

int is_safe(int seq[P])
{
   int work[R];
   int finish[P] = {0};
   int count = 0;

   memcpy(work, avail, sizeof(work));

   while (count < P)
   {
      int found = 0;
```

```
        for (int p = 0; p < P; p++)
        {
            if (finish[p])
                continue;

            int possible = 1;

            for (int r = 0; r < R; r++)
            {
                int need = maxm[p][r] - alloc[p][r];

                if (need > work[r])
                {
                    possible = 0;
                    break;
                }
            }

            if (possible)
            {
                for (int r = 0; r < R; r++)
                    work[r] += alloc[p][r];

                finish[p] = 1;
                seq[count++] = p;
                found = 1;
                break;
            }
        }

        if (!found)
            return 0;
    }

    return 1;
}

void request(int p, const int req[R])
{
    for (int r = 0; r < R; r++)
    {
        int need = maxm[p][r] - alloc[p][r];
```

```
        if (req[r] > need || req[r] > avail[r])
        {
            printf("P%d request: must wait (not available)\n", p);
            return;
        }
    }

    for (int r = 0; r < R; r++)
    {
        avail[r] -= req[r];
        alloc[p][r] += req[r];
    }

    int seq[P];

    if (is_safe(seq))
    {
        printf("P%d request: granted, safe sequence", p);

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);

        printf("\n");
    }
    else
    {
        for (int r = 0; r < R; r++)
        {
            avail[r] += req[r];
            alloc[p][r] -= req[r];
        }

        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main(void)
{
    int seq[P];

    if (is_safe(seq))
```

```
    {
        printf("initial state safe:");

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);

        printf("\n");
    }
    else
    {
        printf("Initial state unsafe\n");
        return 1;
    }

    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});

    return 0;
}
```

OUTPUT:

```
Output

initial state safe: P1 P3 P0 P2 P4
P1 request: granted, safe sequence P1 P3 P0 P2 P4
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

8. Why is P0 request (0, 2, 0) unsafe?

After temporarily allocating the request:

$$Available=(3,1,2)$$

The updated Need matrix does not allow P0, P1 or P2 to finish immediately.

P3 can finish, producing:

$$Work=(5,2,3)$$

P1 can then finish:

$$Work=(8,2,5)$$

P2 can finish:

$$Work=(11,2,7)$$

However, P0 still needs (7,4,3), and P4 needs (4,3,1). Neither can finish

because the available B resource is only 2.
Therefore, the system is unsafe and the request must be denied.

D. Predict Before Implementing
The Banker's safety algorithm checks whether processes can finish using currently available resources.
Let:

- n = Number of processes.
- m = Number of resource types.

For each process, the algorithm checks all resource types and may scan all processes repeatedly.
Worst-case time complexity:
$$\boxed{O(n^2m)}$$
Space complexity:
$$\boxed{O(nm)}$$
for the allocation, maximum and need matrices.

E. Measure the Safety Algorithm (1 Mark)
The assignment requires testing 10, 100 and 1,000 processes.
The following Linux commands can measure the execution time:
gcc bankers.c -o bankers

time ./bankers
For the experiment, create test matrices for 10, 100 and 1,000 processes with three resource types. Use the same safe-state generation method for every size.
Record your actual results:

Number of processes	Resource types	Execution time
10	3	Measure
100	3	Measure
1000	3	Measure

Evaluation

- Increasing the number of processes increases the number of safety checks.
- Increasing the number of resource types increases the resource comparison operations.
- Execution time increases as the number of processes increases.
- The results should approximately follow the predicted $O(n^2m)$ worst-case growth, although actual timings depend on the test data and hardware.

Note: Do not use the small five-process program above as the 10/100/1,000-

process benchmark. It must be extended with generated matrices to perform that experiment correctly.

F. Defend the Final Design

Recommended strategy: Deadlock Avoidance using Banker's Algorithm

Reasons:

1. Prevents allocation that leads to an unsafe state.
2. Protects critical laboratory resources.
3. Ensures processes can complete in a safe sequence.
4. Reduces the possibility of deadlock.
5. Supports controlled resource allocation.

Limitations

- Maximum resource requirements must be known in advance.
- Repeated safety checks increase processing overhead.
- Large systems require more computation.
- Some resources may remain unused to maintain safety.

Alternative – Detection and Recovery

If detection and recovery are used:

- The system periodically checks for deadlocks.
- A deadlocked job is selected for termination.
- Its resources are released.
- Other waiting jobs can continue.

Comparison:

Banker's algorithm	**Detection and recovery**
Checks before allocation	Checks after allocation
Avoids unsafe states	Detects existing deadlocks
Requires maximum demand	Does not require maximum demand in advance
Adds request-time overhead	Adds periodic detection overhead
Avoids losing completed work	Aborting jobs may lose work

Final recommendation: Use the Banker's algorithm for critical laboratory resources when maximum resource demands are known. If maximum demands cannot be predicted, use deadlock detection and recovery with controlled job termination.